



ISO 9001: 2015
SOCOTEC SCP000722Q

REPUBLIC OF THE PHILIPPINES
BICOL UNIVERSITY
POLANGUI
COMPUTER STUDIES DEPARTMENT
Polangui, Albay



IT112 2s2526 - WEB SYSTEMS AND TECHNOLOGIES

Phase 7: Testing, Documentation, and Final Deployment

FINAL LAB DOCUMENTATION

Uniformity: An E-Commerce platform for Pre-Loved School and Professional Uniforms



GROUP NAME: TechNic Course & Block: BSIT-2A		
MEMBERS	ROLE	SIGNATURE
Cipriano, Mcxyron B.	<i>Back-End Developer</i>	
Hernandez, Kurt Jushua S.	<i>Database Manager/Tester & Debugger</i>	
Registrado, Mharie Franz	<i>Tester & Debugger</i>	
Red, Paul Orlando B.	<i>Project, Database, GitHub Manager & Documentation Officer</i>	
Romano, Jay L.	<i>Front-End Developer</i>	

Submitted to:

Guillermo V. Red, Jr., DIT

Assistant Professor IV

I. System Overview and Objectives

Uniformity is a full-stack web-based marketplace designed for buying and selling pre-loved school/professional uniforms. The system was created to help students find affordable uniforms while also allowing other students to sell uniforms that are still in good condition. Instead of relying on informal posting, manual messaging, or unorganized transactions, Uniformity provides a structured platform where products, carts, orders, sellers, buyers, and administrators are managed in one system.

The main objective of the project is to demonstrate the complete development of a deployed full-stack web application using frontend, backend, database, authentication, CRUD operations, responsive design, and Progressive Web Application concepts. The system also aims to support real marketplace workflows such as product listing, cart management, checkout, seller fulfillment, buyer order confirmation, and admin management.

II. Features Implemented

- Uniformity includes user account registration and login with **JWT-based authentication**.
- Users can create an account, log in securely, and access **protected pages**.
- The system also allows users to **edit their profile information**, including their name, email, and phone number.

For buyers:

- The system includes **product browsing**, product search, category and size filtering, sorting, pagination, and product detail viewing.
- Buyers can **view product details** including images, descriptions, sizes, prices, seller information, and related product suggestions.
- A **contact seller section** that shows seller contact information.
- The **cart feature** allows buyers to add products to their cart, view all cart items, adjust quantities, remove items using a confirmation modal, and select which items to checkout.
- **Checkout page** allows buyers to provide delivery information, including name, phone number, and address details.

- The **checkout logic separates orders by seller**. If a buyer checks out products from two different sellers, the system creates separate orders. If products are from the same seller, they remain grouped under one order.

For orders:

- **My orders page** to view user's order history, open order details, cancel pending orders, confirm received orders, and delete delivered or cancelled order history.

For sellers:

- Includes **product listing management**. Sellers can add listings, edit product details, delete listings, and view incoming orders.
- Sellers can also **view order details** and **mark assigned order items as fulfilled**. Once the buyer confirms receipt, the seller side reflects the delivered status.

For administrators:

- Includes an **admin panel with dashboard statistics and management sections**.
- **Admins can manage** users, products, categories, and orders.
- The admin panel includes **search and filter tools, role badges, edit modals, status controls, and confirmation modals**.

The system also includes responsive design improvements for mobile, tablet, and desktop screens. The UI includes **sticky topbars, a mobile sidebar, modal confirmations, smooth UI states, product image preview overlay**, and cleaner layouts for cart, orders, listings, profile, and admin pages.

Progressive Web App support was added. The system includes a web app manifest, service worker, PWA icons, installability support, and basic offline app-shell caching. Offline mode allows cached static pages and assets to load, but database-driven actions still require internet access.

III. Technologies Used

The frontend was built using **HTML5, CSS3, Bootstrap 5, Bootstrap Icons, and vanilla JavaScript**. The frontend contains static HTML pages, shared styling, page-specific JavaScript files, and API integration through fetch requests.

The backend was built using **Node.js and Express.js**. **MongoDB Atlas** was used as the cloud database, and Mongoose was used for schema modeling and database operations. Authentication was implemented using **JSON Web Tokens**, and passwords are secured using **bcrypt**. **Cloudinary API** is used for product image uploads.

The system uses **Render for deployment**. Git and GitHub were used for version control, branch management, collaboration, and release documentation. Chrome DevTools was used for testing responsiveness, service workers, cache storage, and PWA installability.

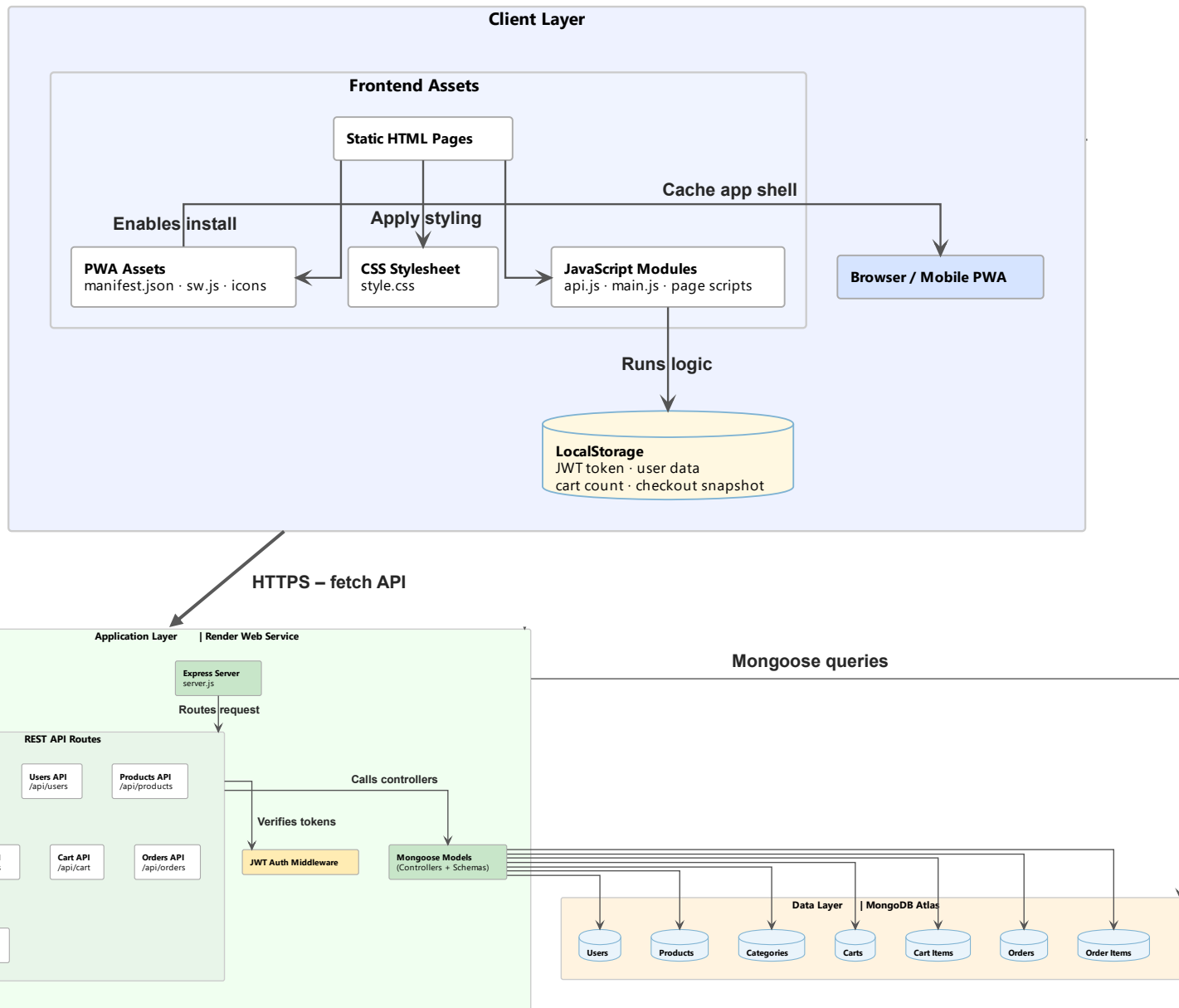
IV. Architecture Diagram (Frontend–Backend–Database flow)

The system follows a frontend-backend-database architecture. The user accesses the system through a browser or mobile PWA. The frontend consists of static HTML pages, CSS styling, JavaScript modules, and PWA files such as manifest.json and sw.js.

The frontend sends HTTP requests using the fetch API to the backend server. The backend is an Express.js application deployed on Render. It serves both the frontend files and the REST API routes. The backend contains route modules for authentication, users, products, categories, cart, orders, and admin functions.

Protected routes use JWT authentication middleware to verify user access. The backend communicates with MongoDB Atlas through Mongoose models. The database stores users, products, categories, carts, cart items, orders, and order items.

Uniformity System Architecture



V. Entity Relationship Diagram (ERD)

The database design includes several main collections. The Users collection stores account information such as **name**, **email**, **password**, **phone number**, **role**, **status**, and **date created**. A user can act as a **buyer**, **seller**, or **admin** depending on the system flow and assigned role.

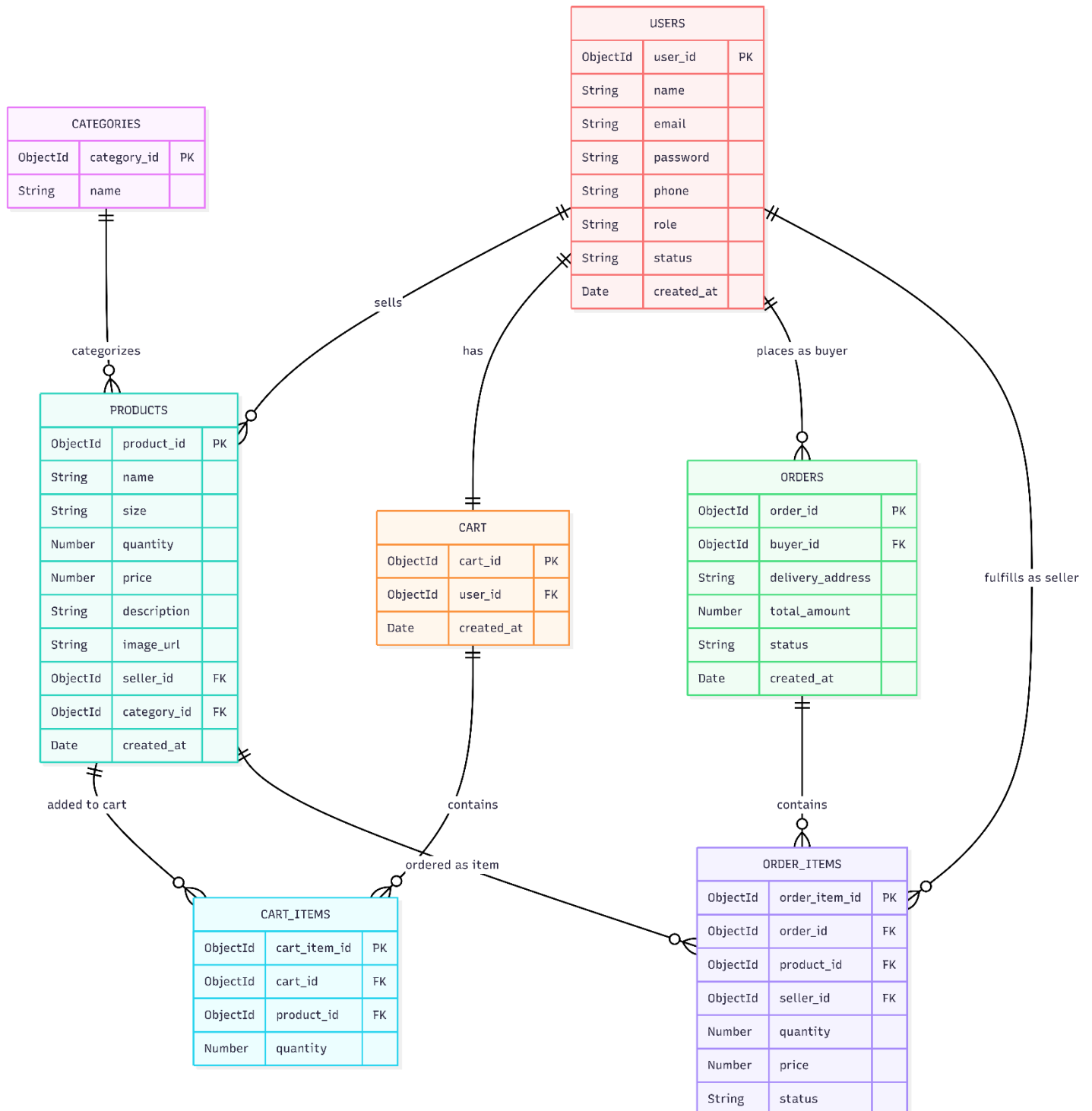
The Products collection stores listing details such as **product name**, **size**, **quantity**, **price**, **description**, **image URL**, **seller reference**, **category reference**, **status**, and **creation date**. Each product belongs to one seller and one category.

The Categories collection stores **product category names** used for filtering and organizing listings.

The Cart collection belongs to a user. Cart Items connect a cart to products and **store the quantity** selected by the buyer.

The Orders collection stores **buyer order data, delivery address, total amount, order status, and date created**. The Order Items collection stores the specific products inside each order, including product reference, seller reference, quantity, price, and item status.

The order system separates orders by seller. This means one checkout can create multiple orders if the selected items belong to different sellers.



VI. GitHub Link + Commit Logs Summary

GitHub Repository Link: <https://github.com/red5458/final-project-TechNic-BSIT2a>

The GitHub commit history shows the development of the system from planning to final release. Early commits focused on setting up the project structure, backend folders, frontend folders, and initial documentation.

Commits on Mar 26, 2026		
Restored project folder structure	red5458 committed on Mar 26	d216cf5  
Removed .gitkeep files from backend, database, docs/planning, and frontend directories	red5458 committed on Mar 26	d7083fb  
Added basefolders with .gitkeep files so that the basefolder can be committed to the repository	red5458 committed on Mar 26	759fa5d  
Added Readme File	red5458 committed on Mar 26	1ca0a4e  
Commits on Mar 28, 2026		
Added ERD Diagram	Kurt13Hernandez committed on Mar 28	9c29974  
Added info on Readme file	mhariefrazregistrado25 committed on Mar 28	22b8f10  
Added back the Final Project Proposal document with updated cover page	red5458 committed on Mar 28	1852af7  
Commits on Apr 11, 2026		
Add database connection setup using mongoose	red5458 committed last month	86bb238  
Commits on Apr 10, 2026		
Updated ERD diagram image & in Final Project Proposal document in planning directory	red5458 committed last month	9cd5506  
Commits on Apr 10, 2026		
Added Git ignore	red5458 committed on Apr 10	6123561  

Backend commits included Express server setup, MongoDB connection, Mongoose models, authentication routes, product routes, cart routes, order routes, and admin routes.

Add Mongoose models for Cart, CartItem, Category, Order, OrderItem, Product, and User	red5458 committed last month	81d8e2d  
Merge pull request #1 from red5458/database-manager	red5458 authored last month	Verified 5fe047b  
Add database connection setup using mongoose	red5458 committed last month	86bb238  
Commits on Apr 10, 2026		
Added server.js	Xye1110 committed last month	34327a5  
Added routes	Xye1110 committed last month	b9a150c  
Added controllers	Xye1110 committed last month	25963fb  
Merge pull request #7 from red5458/database	red5458 authored last month	Verified 5954b52  
Merge pull request #6 from red5458:backend	red5458 authored last month	Verified 16337e0  
Add seed script for categories in the database	red5458 committed last month	d6de5fc  

Frontend commits included the landing page, login and register pages, dashboard, product detail page, cart page, checkout page, profile page, my orders page, my listings page, and admin panel pages.

Merge pull request #10 from frontend branch 🔒			Verified	6f29a30		
red5458 authored 3 weeks ago						
added api.js and main.js to frontend/js folder	Clyve117 committed 3 weeks ago			e3bd65f		
added logo.png to frontend/img folder	Clyve117 committed 3 weeks ago			3f9e5a5		
added style.css to frontend/css folder	Clyve117 committed 3 weeks ago			698a0fc		

added cart.html to frontend folder	Clyve117 committed 3 weeks ago			2c17d9c		
added dashboard.html to frontend folder	Clyve117 committed 3 weeks ago			e049052		
add login.html to frontend folder	Clyve117 committed 3 weeks ago			a8f7248		

Later commits focused on API integration, form submissions, dynamic rendering, cart logic, checkout logic, order separation by seller, seller fulfillment, buyer received confirmation, and admin management functions.

Merge pull request #11 from Phase-4-Link-HTML-forms-to-the-corresponding-backend-routes 🔒			Verified	996ec9c		
red5458 authored 3 weeks ago						
Implement edit profile form submission with validation and error handling	Xye1110 committed 3 weeks ago			c68bcf9		
Implement add to cart functionality with error handling and localStorage update	Xye1110 committed 3 weeks ago			ab836be		
Implement checkout form submission with validation and error handling	Xye1110 committed 3 weeks ago			4318599		

Linked api.js in all HTML files to enable API calls and enable data flow	red5458 committed 3 weeks ago			b50c92e		
Added the frontend screenshots to the lab11 folder in docs directory	red5458 committed 3 weeks ago			c1c714f		

Merge pull request #13 from Phase 5 branch 🔒			Verified	ec429d3		
red5458 authored 2 weeks ago						
Updated scripts for improved data fetching and rendering across various pages.	red5458 committed 2 weeks ago			637f884		
Implement to display profile details, stats, and recent orders in profile.js	red5458 committed 2 weeks ago			030a5ba		
Implement product detail fetching and rendering in product-detail.js	red5458 committed 2 weeks ago			e98fa53		

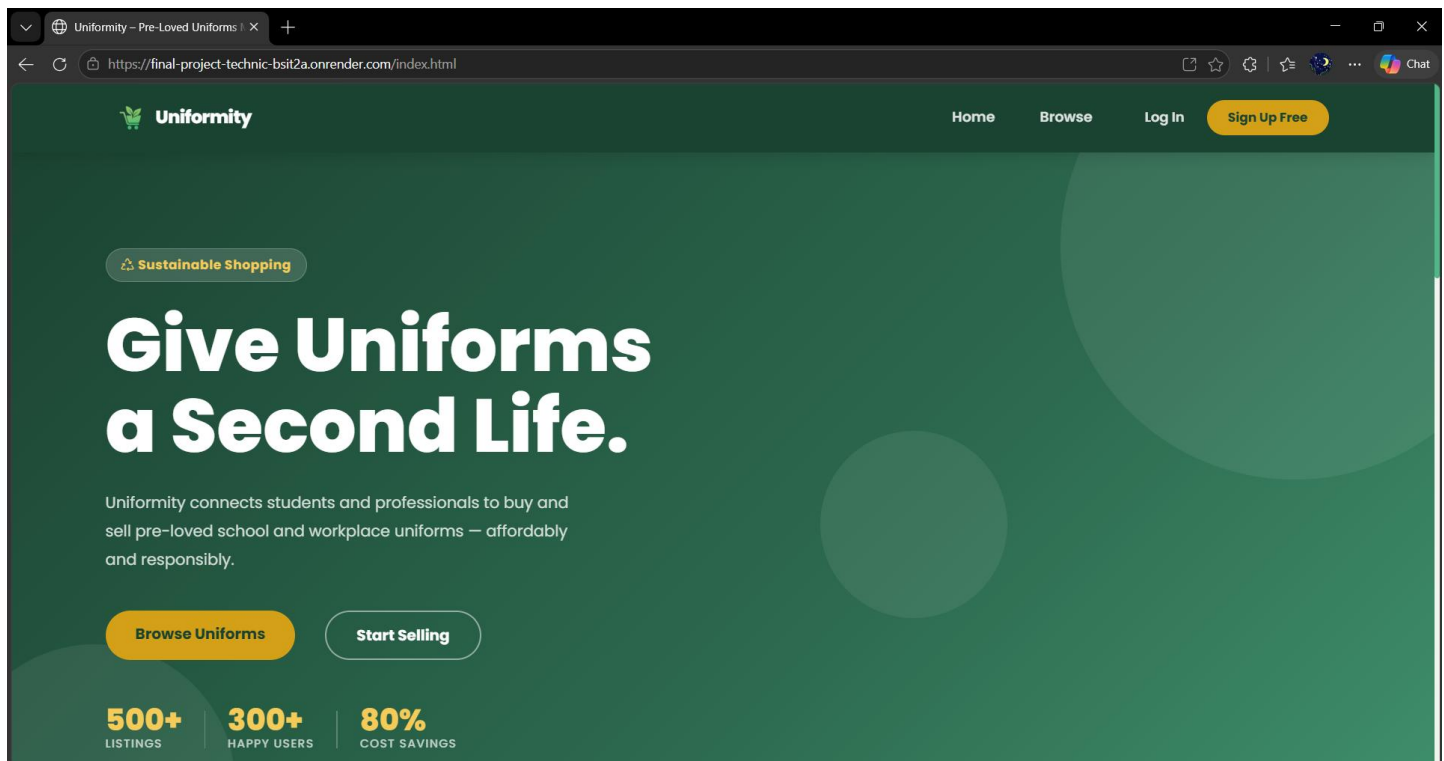
Several commits were dedicated to UI improvements, responsiveness fixes, modal confirmations, sticky topbars, sidebar behavior, cart layout, profile layout, order cards, listing tables, and admin panel polish. Final commits included PWA implementation, service worker caching, manifest setup, README updates, documentation cleanup, and final bug fixes.

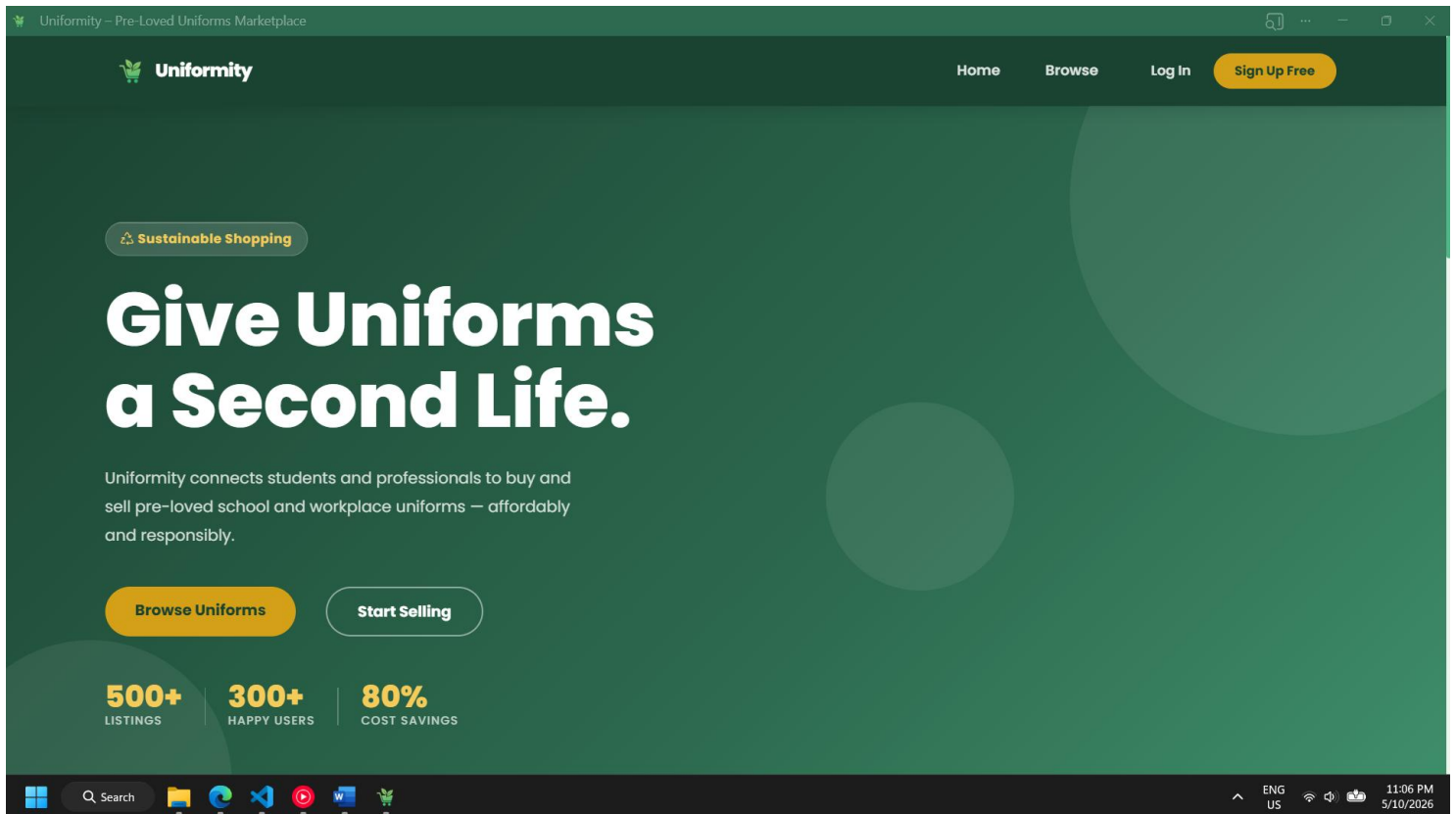
Refactor category controller: improve structure and ensure consistent responses	d4995e7	<>
Refactor auth controller: add clarity and reuse comment	ef579a9	<>
Refactor cart controller: enhance error handling and optimize item retrieval logic	8098f5c	<>
Remove unnecessary comment line in productController.js	15f294c	<>

Updated root Readme	ad4c42f	<>
Update README with project overview, features, and installation instructions; enhance cart functionality and order fulfillment logic in JavaScript files; increment service worker cache version.	19df755	<>
Add PWA support with manifest and service worker (#21) xxx	Verified d22e899	<>
Profile order UI polish (#20) xxx	Verified e390a4b	<>
Merge Admin panel branch (#18) xxx	Verified e315b97	<>
Commits on May 9, 2026		
Phase 7 testing documentation and final deployment (refinement) (#17) xxx	Verified 1cf9193	<>

VII. Render Link / Screenshots

Live Render Link: <https://final-project-technic-bsit2a.onrender.com>





VIII. Summary of Division of Tasks

Mcxyron B. Cipriano - contributed as the back-end developer. His tasks focused on backend development, API routes, database logic, and server-side functionality.

Jay L. Romano - contributed as the front-end developer. His tasks focused on frontend page layouts, interface implementation, and visual structure.

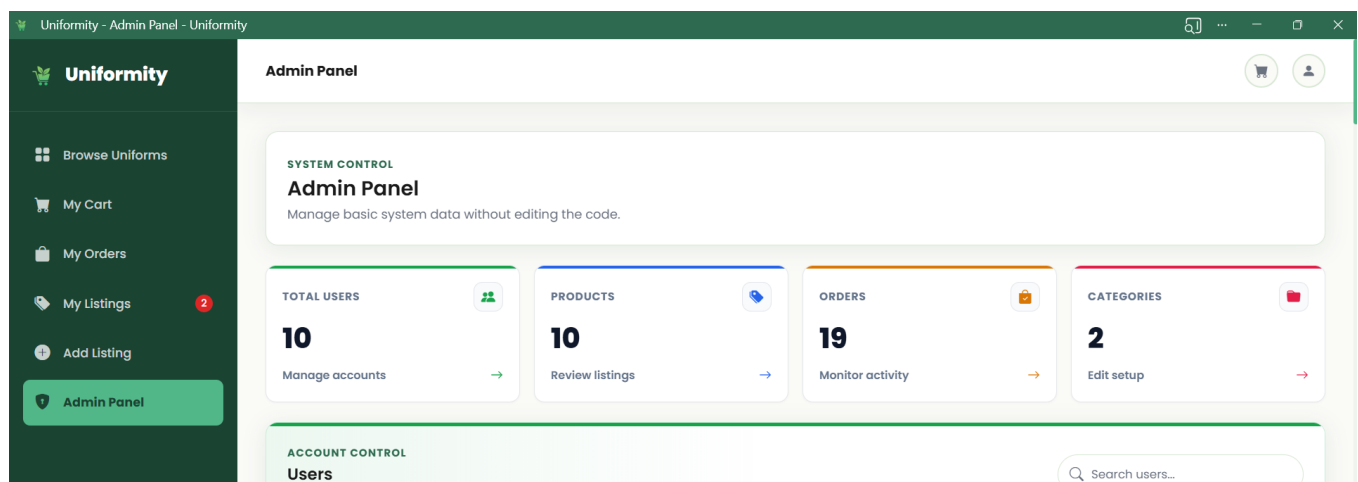
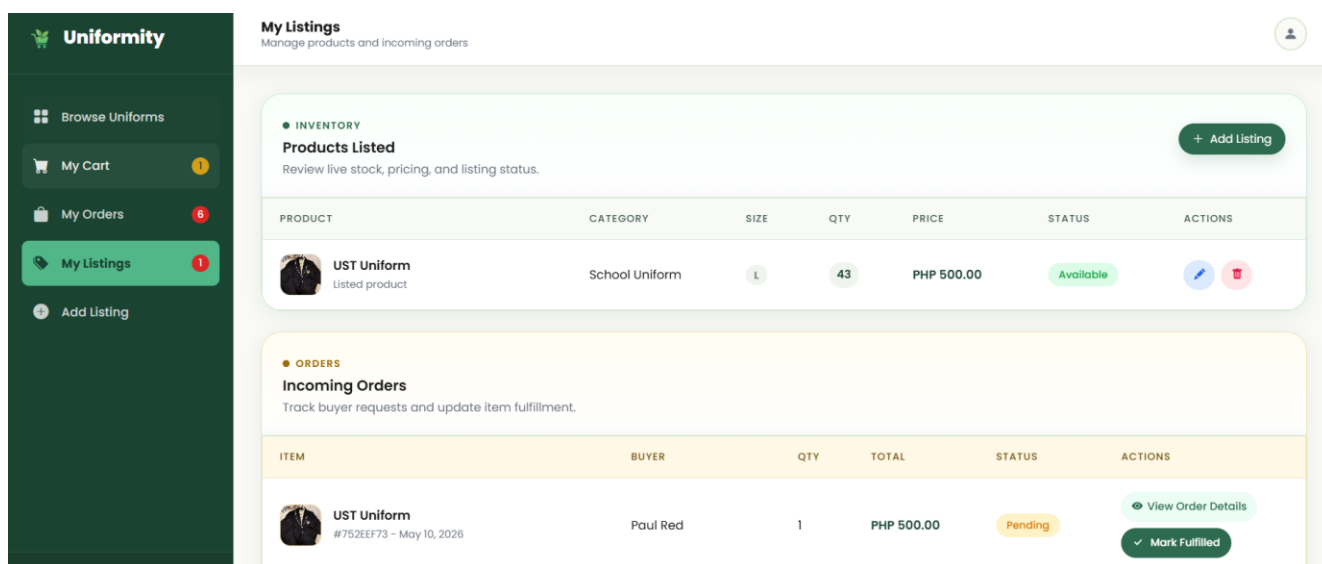
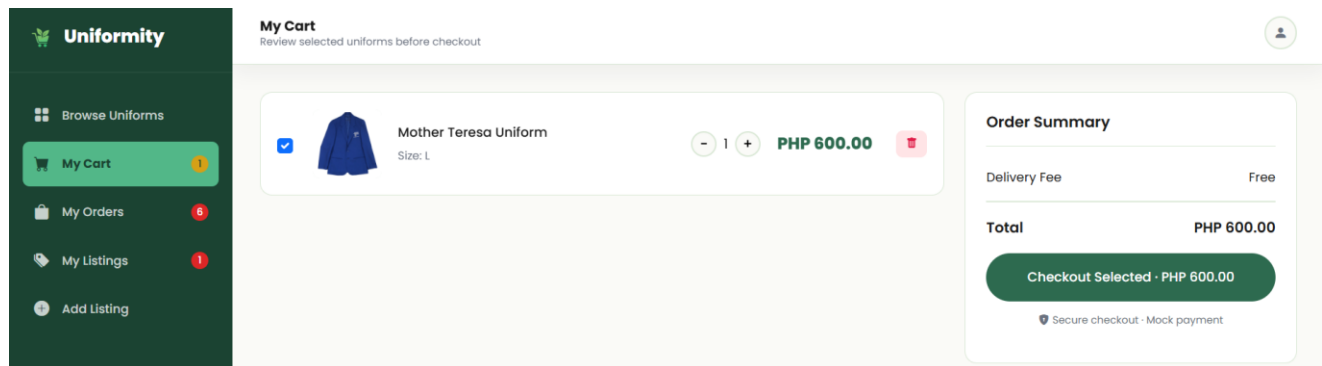
Paul Orlando B. Red - contributed as project manager, database manager, GitHub manager, and documentation officer. His tasks included coordinating the group workflow, managing branches and merges, reviewing code, integrating frontend and backend features, fixing bugs, improving UI responsiveness, polishing the interface, updating documentation, and preparing final project materials.

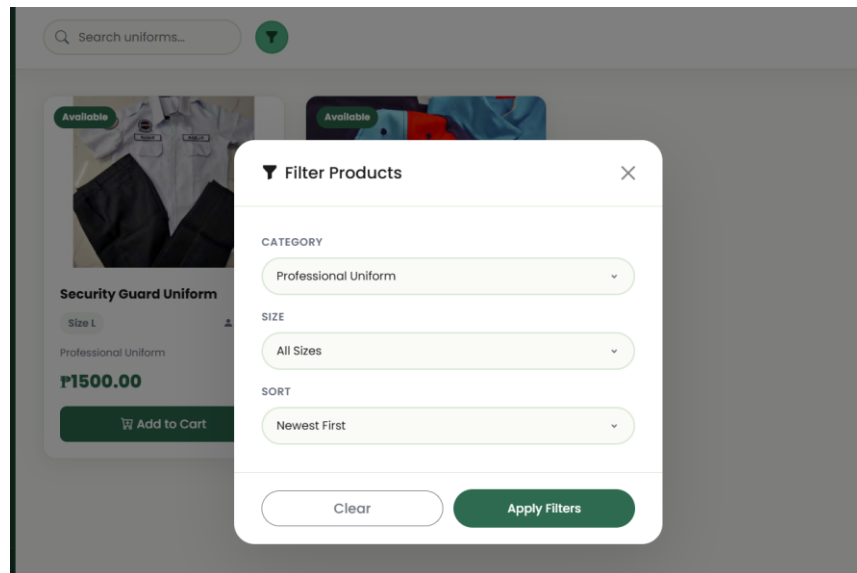
Kurt Jushua S. Hernandez - contributed as database manager, tester, and debugger. His tasks included in creating the models, supporting database-related work, testing features, and helping identify issues during development.

Mharie Franz Registrado - contributed as tester. Her tasks included intensive testing of system flows, checking user-facing features, and helping identify errors before final submission on each phases until the final.

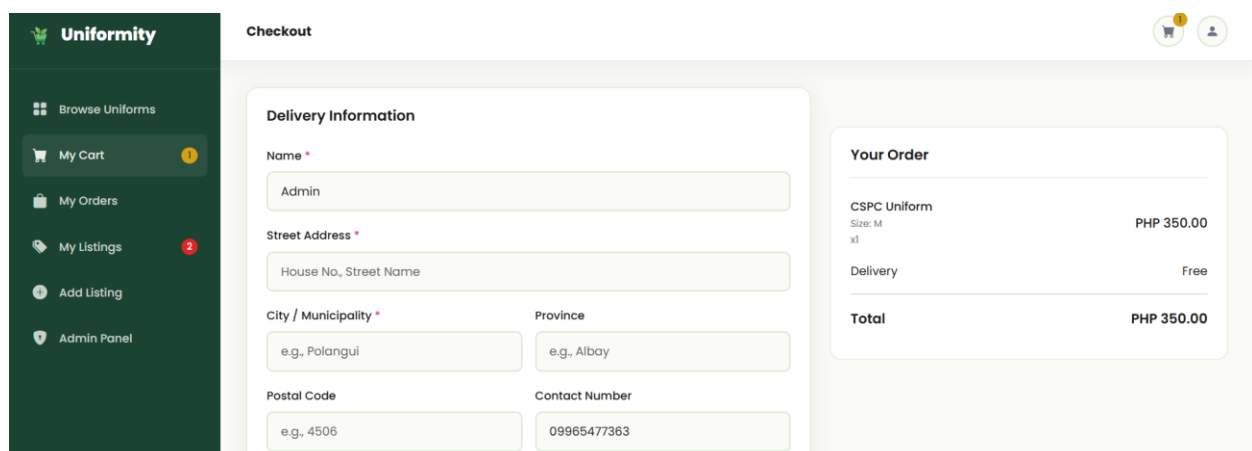
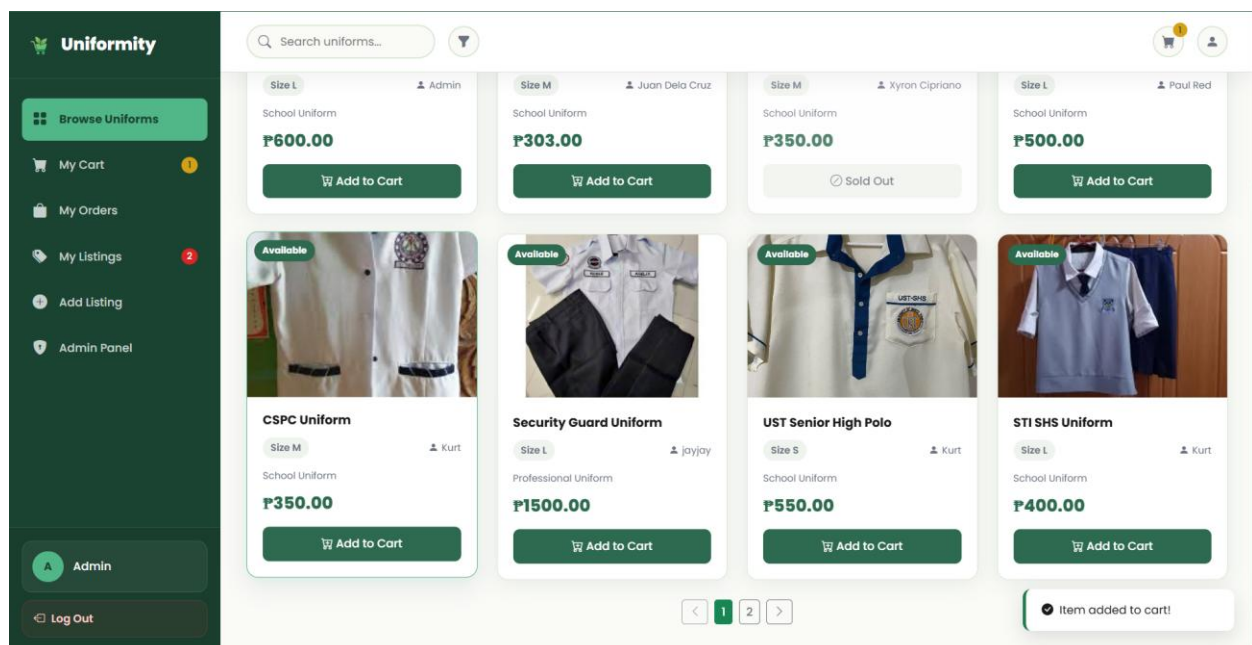
IX. Bug testing results / final test notes

The system was tested across the main buyer, seller, and admin workflows. Registration and login were tested to confirm that authentication and protected pages work correctly. Product browsing was tested with search, filters, sorting, pagination, and product detail viewing.

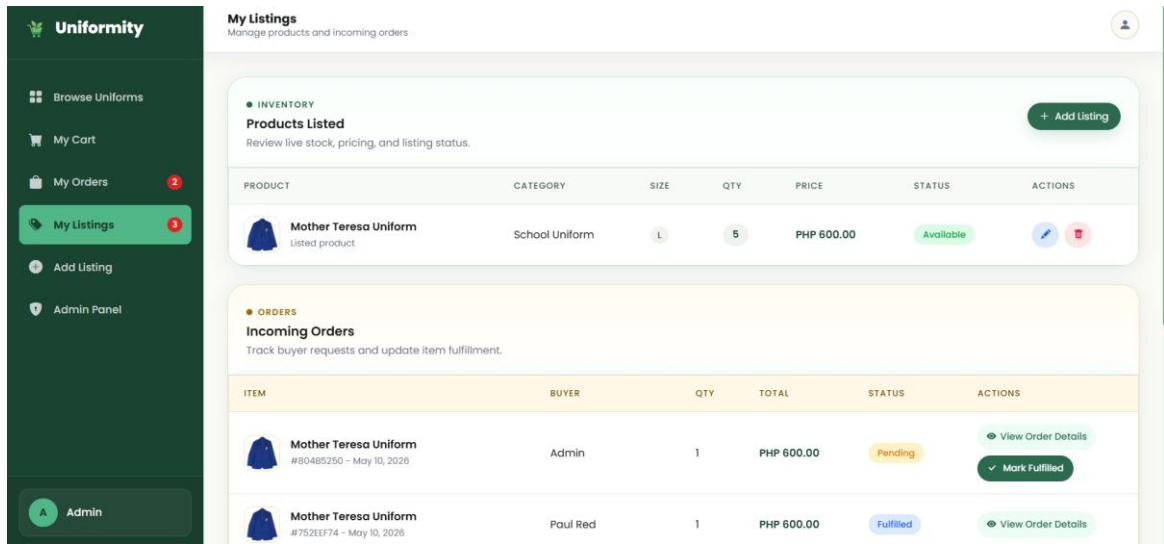
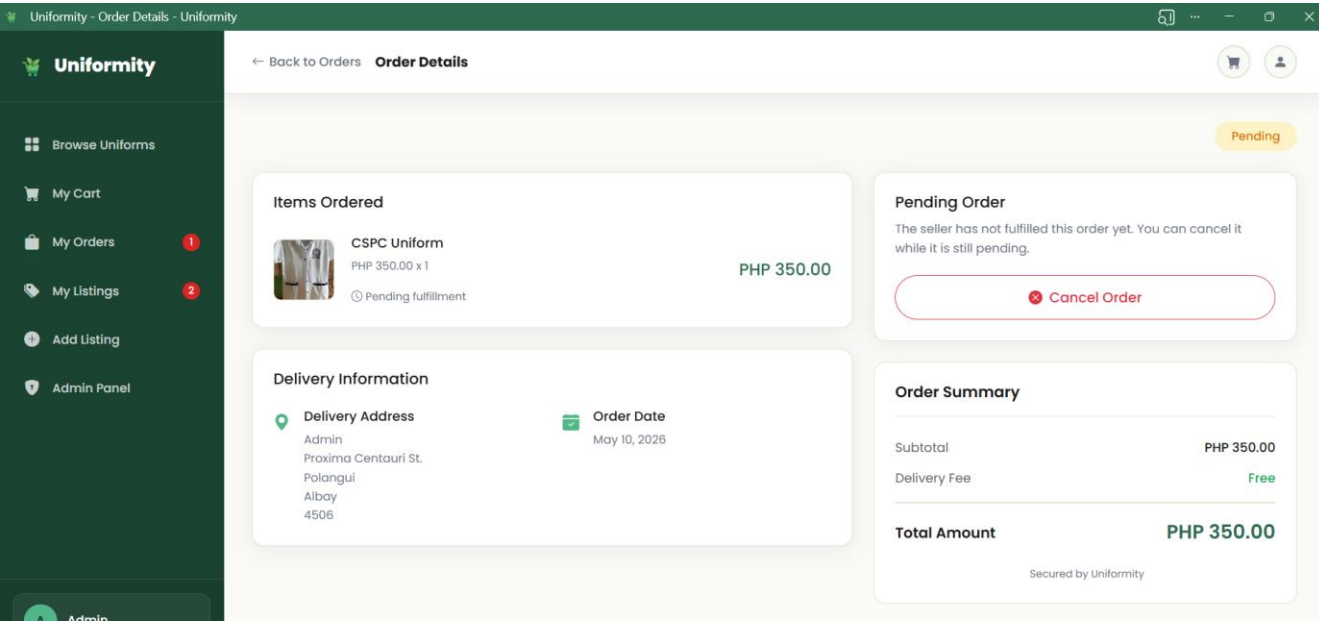
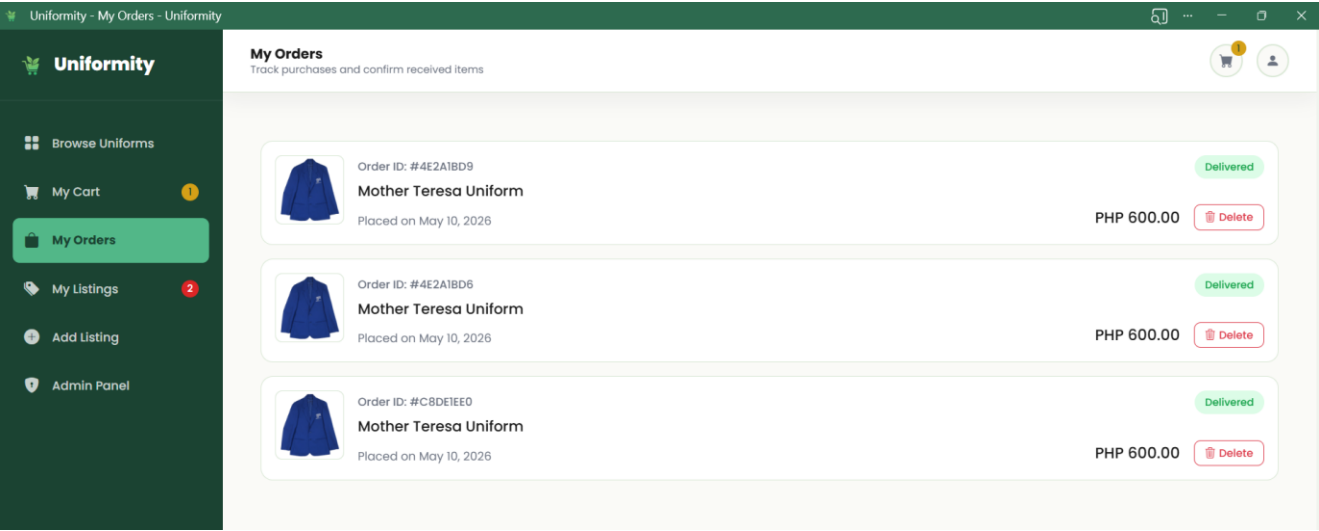


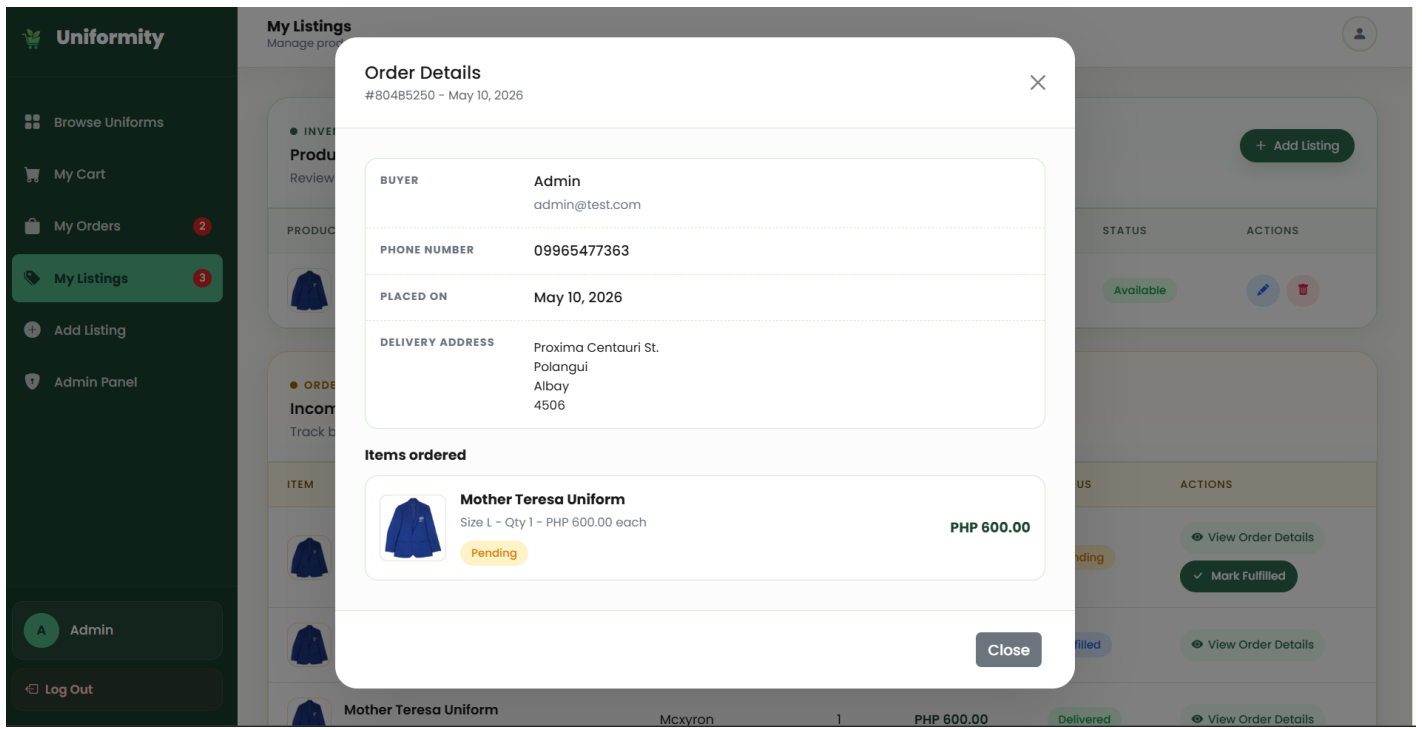


Cart testing included adding products, updating cart badge count, changing quantities, deleting cart items, and checking out selected items. The checkout flow was tested to confirm that delivery details are saved and that orders are separated correctly when products come from different sellers.

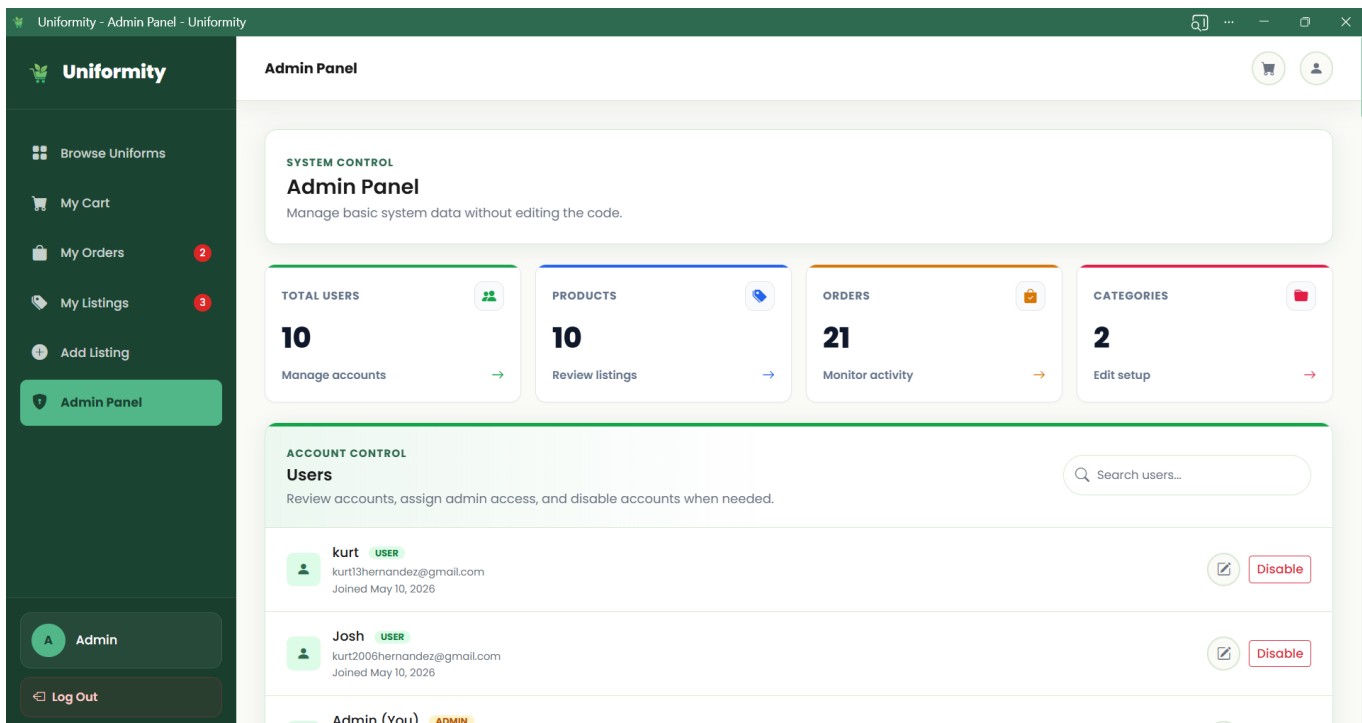


Buyer order testing included viewing order history, opening order details, cancelling pending orders, confirming received orders, and deleting delivered or cancelled order history. Seller testing included adding listings, editing listings, deleting listings, viewing incoming orders, viewing order details, and marking items as fulfilled.

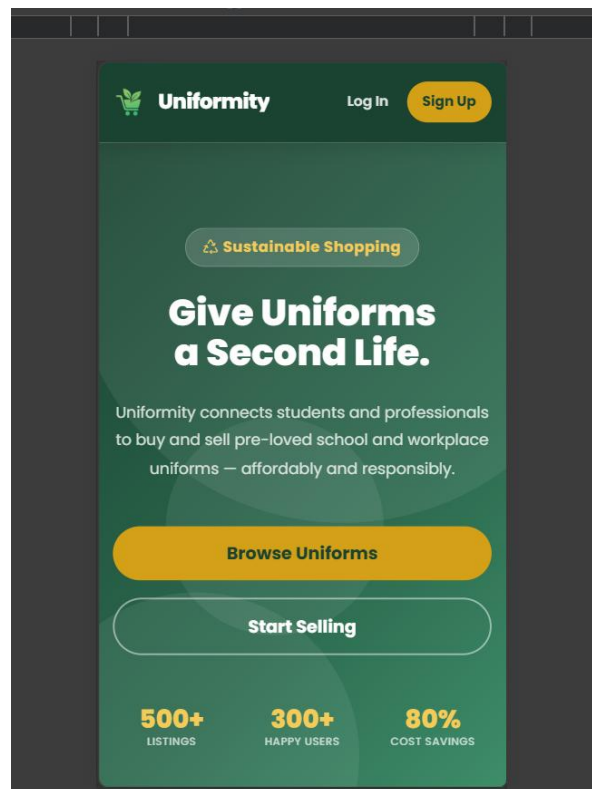
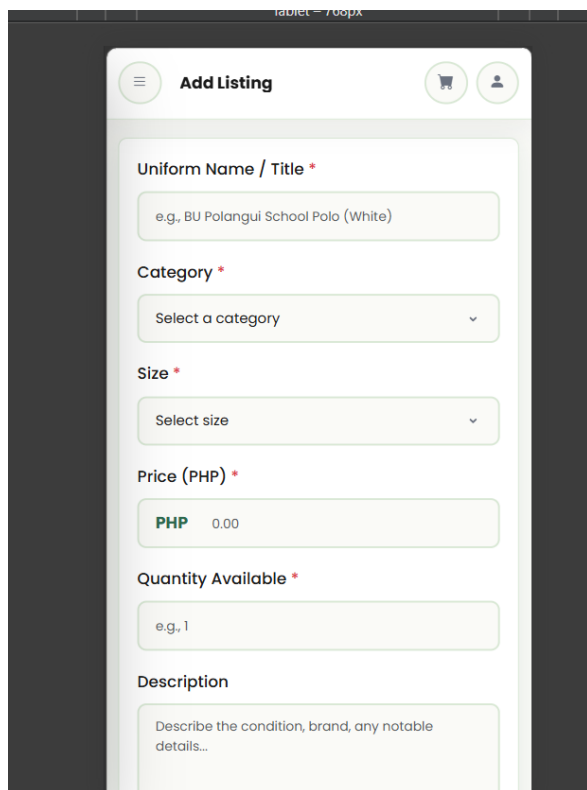
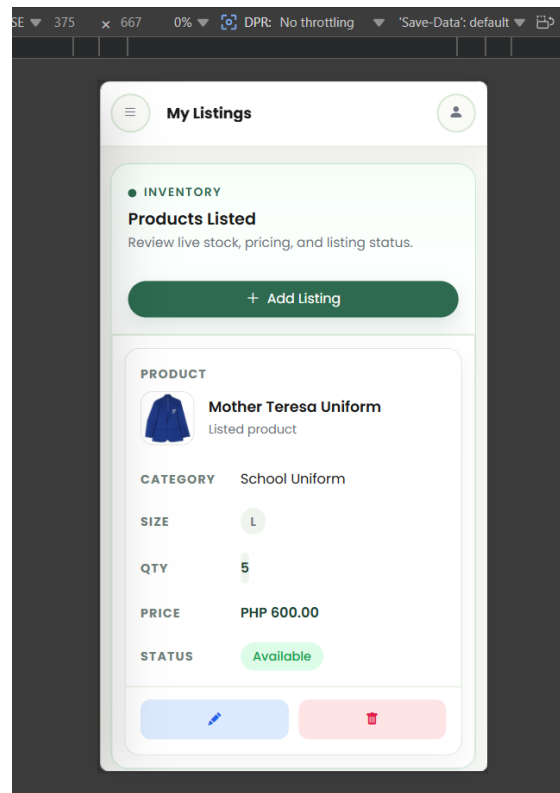
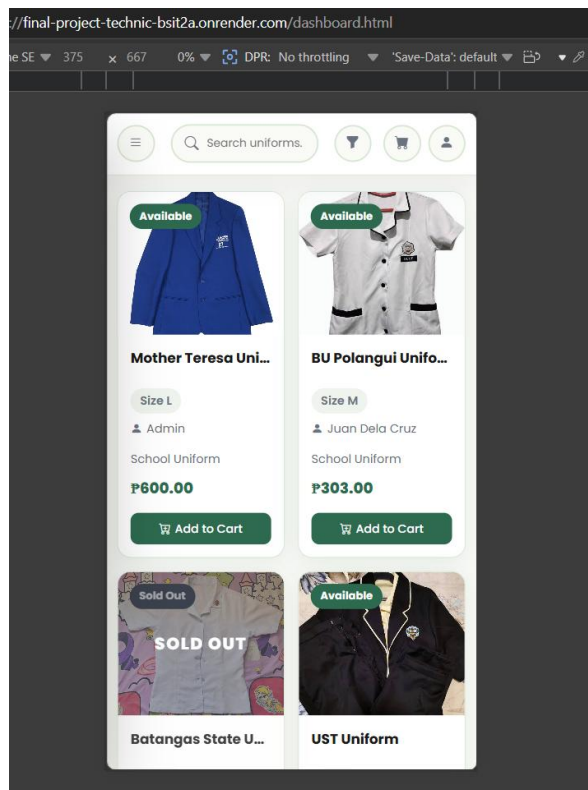




Admin testing included managing users, products, categories, and orders. Search and filter tools were also tested inside the admin sections. Status updates were checked to make sure the UI updates without unnecessary full refresh behavior.



Responsiveness testing was done on smaller screen widths and mobile devices. Several layout issues were fixed in the cart page, my listings page, my orders page, profile page, product detail page, dashboard filters, topbars, and sidebar. PWA testing included checking the manifest, service worker registration, cache storage, installability, and basic offline loading.



Final test notes showed that the system is stable for the required academic project scope. Live database actions still require internet connection, while offline mode supports cached static app-shell loading only.

X. Short reflection from each member

❖ Mcxyron B. Cipriano:

For this final phase, I contributed by helping check the backend side of the system and making sure the API-related functions were still working after the final changes. I also helped review if the database-connected features such as products, orders, cart, and user data were still functioning properly during final testing. This phase helped me see how important final checking is before submission, because even small changes in the frontend or deployment setup can affect how the backend behaves.

❖ Jay L. Romano:

I helped review the frontend side of the system, especially in this last phase, we really test it very carefully - how the pages looked and behaved after the final updates. I checked if the layouts were optimized on smaller screens. This week made me realize that frontend work does not end after creating the pages, because the final adjustments, spacing, responsiveness, and consistency still matter a lot for the overall presentation.

❖ Paul Orlando B. Red:

This final phase was mostly about putting everything together and making sure the project was ready for submission. I worked on the remaining UI fixes with my frontend dev like cart badge issue, seller order update behavior, & PWA setup. Also did the README revision, diagrams, and final documentation content. I also checked the system after deployment and made sure the final requirements were reflected in the documents. Compared to the earlier phases, this part felt more like harder but also fulfilling that we as a group — successfully finished this project. This allows me to gain experiences as a project manager and also a developer.

❖ Kurt Jushua S. Hernandez:

I contributed to database in adding user roles in models, testing, and debugging. I helped check if the system features were working properly and assisted in identifying issues with my fellow tester —Mharie during this last phase. This project helped me understand the importance of testing and database consistency.

❖ **Mharie Franz Registrado:**

My contribution in this phase was mainly on final checking and user-side testing. I helped look at the system for possible errors, confusing layouts, and parts that might not work well during presentation and report it to our manager. Since the project was already close to completion, this phase showed me how important it is to test the system carefully before submitting it, because small issues can still affect how the project is judged.

Note: Approximately 40% of AI assistance was used in preparing and refining the project documentation. I used it to help me organize sections, improve wording, summarize implemented features and format technical explanations. However, the contents were reviewed, edited, verified, and finalized by me as also the project manager based on the actual system implementation in our overall project development.